

# ***Graph-based Diffusion Solvers for Combinatorial Optimization***

백승언, 펀디멘탈팀

12 May, 2024

- **Introduction**

- Combinatorial Optimization(CO) problem
- Machine learning approaches to CO problem

- **DIFUSCO**

- Backgrounds
- Methodology

- **Experimental results**

# Introduction

# Combinatorial optimization problems (I)

## ● What is Combinatorial Optimization(CO) problem?

- Finding an optimal objects from a finite set of objects<sup>[1]</sup>
- In many hard CO problems, exhaustive search is not tractable
  - Optimal solution require exponential computing time
- Conventional CO problems generally rely on handcrafted heuristics or optimization methods
  - Once the problem statement changes slightly, they need to be revised and this processes are time-consuming

## ● Previous machine learning approaches to CO problem

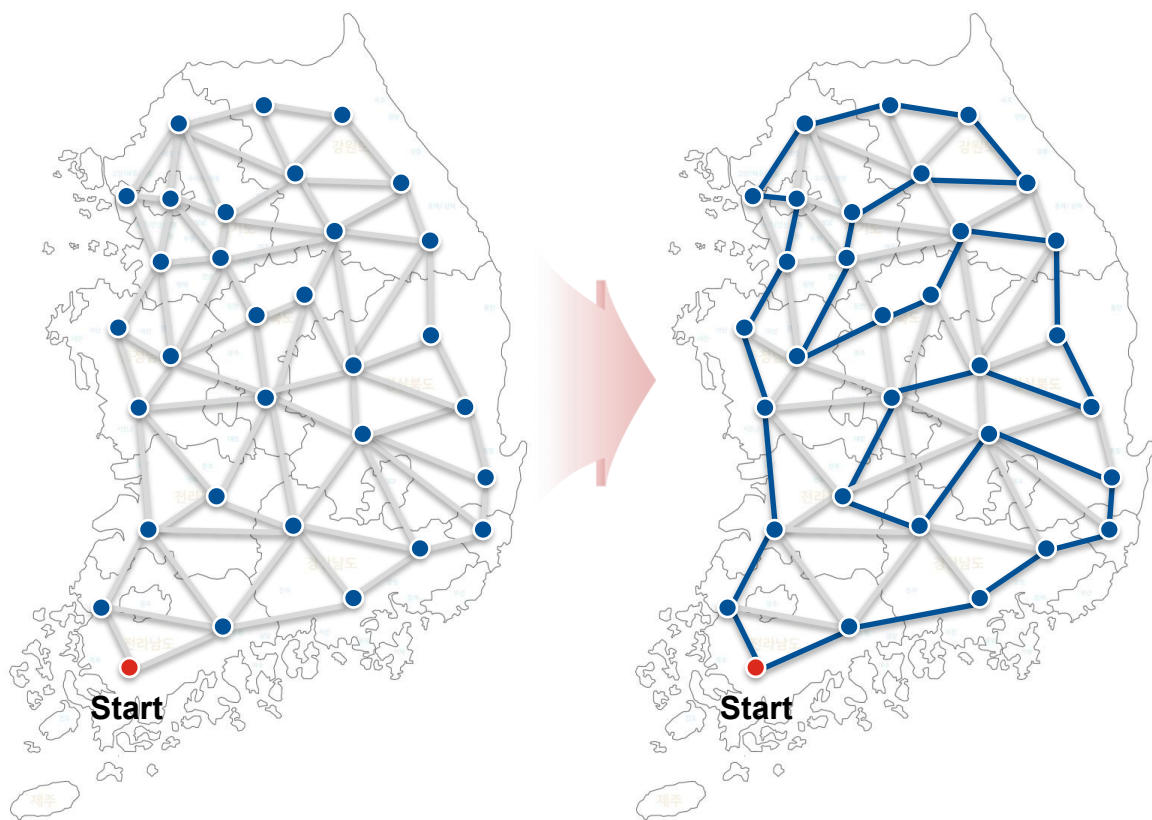
- Supervised learning does not apply to most CO problems because it is difficult to acquire optimal labels
  - Exceptionally, Pointer Network beats the CO problem with small problems
- Many studies show that the Reinforcement Learning(RL) have potential to tackle the CO problems
  - Message passing neural networks with value-based reinforcement learning
  - Attention-based RNN with policy-gradient reinforcement learning

[1] : [https://en.wikipedia.org/wiki/Combinatorial\\_optimization](https://en.wikipedia.org/wiki/Combinatorial_optimization)

# Combinatorial optimization problems (II)

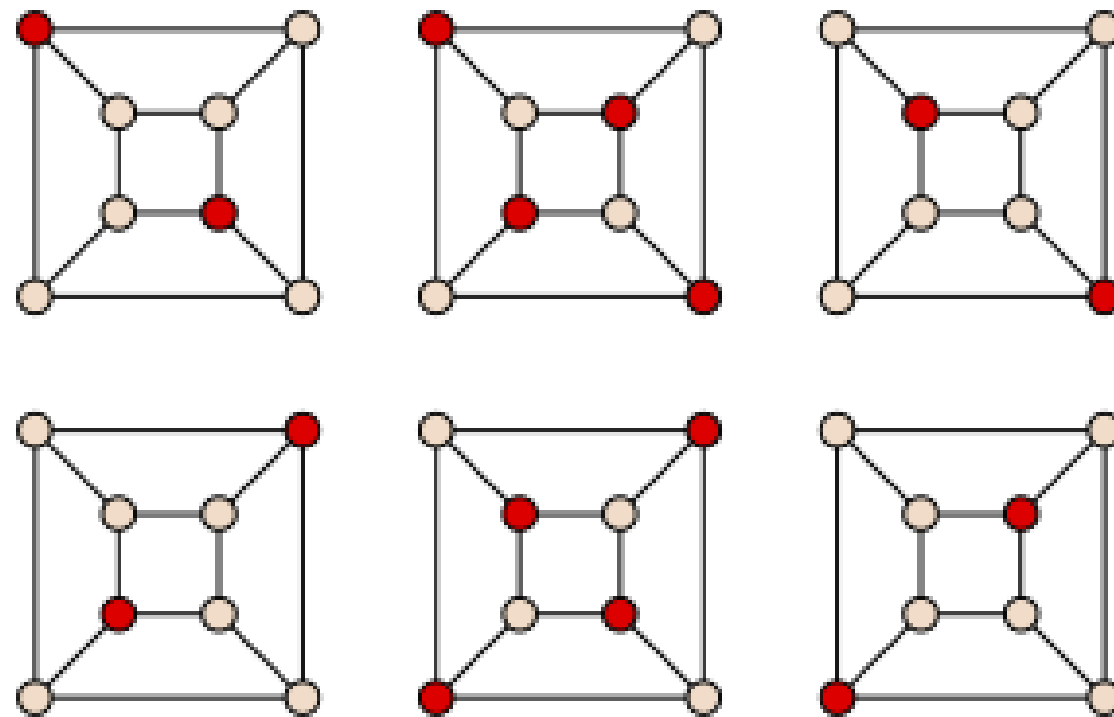
## ● Travelling Salesman Problem(TSP)

- The problem is how a salesman can cover all the cities with minimal cost



## ● Maximal Independent Set(MIS)

- The problem is aim to find independent set that is not a subset of any other independent set



# Machine learning approaches to CO problem

- **Recent development in deep learning has shown new promise in solving CO problems**

- The authors classified the existing neural CO solvers into three categories based on how the solutions are generated
  - The **autoregressive constructive solvers**, the **non-autoregressive constructive solver**, and the **improvement heuristics solver**
- Autoregressive constructive solver
  - They generally used autoregressive factorization to sequentially grow a valid partial solution
  - Those methods typically suffer from the costly computation in their sequential decoding parts and hence are difficult to scale up to large problems
- Non-autoregressive constructive solver
  - They relied on non-autoregressive modeling for scaling up, with a conditional independence assumption among variables
  - Such an assumption, however, unavoidably limits the capability of those methods to capture the multimodal nature of the problems, for example, when multiple optimal solution exists for the same graph
- Improvement heuristics solver
  - They usually used MDP to iteratively refine an existing feasible solution with neural network-guided local operations such as 2-opt and node swap
  - These methods have also suffered the difficulty in scaling up and the latency in inference

**DIFUSCO**

- **Recent remarkable success of diffusion has demonstrated new promise in solving CO problems**
  - Discrete diffusion models have been proposed for the generation of discrete image bits or texts using binomial noises and multinomial/categorical noises
    - Recent research has also shown the potential of discrete diffusion models in sound generation, protein structure generation, molecule generation, and better text generation
  - Graikos et al. proposed an image-based diffusion model to solve Euclidian TSP by projecting each TSP instance onto a  $64 \times 64$  greyscale image space and used a CNN to generate the predicted solution image (TSP)
  - Niu et al. proposed a continuous score-based generative framework for graphs
    - They evaluated simple non-NP-hard CO tasks such as Shortest Path and Maximum Spanning Tree
- **The authors introduced a DIFUSCO, stands for the graph-based diffusion solvers for CO**
  - Such a graph-based diffusion model overcomes the limitations of previous neural solvers
    - First, DIFUSCO could perform inference on all variables in parallel with a few denoising steps, avoiding the sequential generation problem of **autoregressive constructive solvers**
    - Second, DIFUSCO can model a multimodal dist. via iterative refinements, which alleviates the expressiveness limitation of previous **non-autoregressive constructive models**
    - Lastly, DIFUSCO is trained in an efficient and stable manner with supervised denoising, which solves the training scalability issue of **RL-based improvement heuristics methods**



- **Generic framework which is applicable to different NPC problems**

- The authors presented a generic problem definition and discussed the approach to parameterize the problem for mutation so that it can be learned
- Notation
  - $s$ : CO problem instance
  - $\chi_s = \{0,1\}^N$ : the space of candidate solutions  $\{\mathbf{x}\}$  for a problem instance  $s$
  - $c_s : \chi_s \rightarrow \mathbb{R}$ : the objective function for solution  $\mathbf{x} \in \chi_s$ 
    - $c_s(\mathbf{x}) = \text{cost}(\mathbf{x}, s) + \text{valid}(\mathbf{x}, s)$
  - $\text{cost}(\mathbf{x}, s)$ : the task-specific cost of a candidate solution (e.g. the tour length in TSP)
  - $\text{valid}(\mathbf{x}, s)$ : the validation term that returns 0 for a feasible solution and  $+\infty$  for an invalid one
- The optimization objective is to find the optimal solution  $\mathbf{x}^{s*}$  for a given instance  $s$  as:
  - $\mathbf{x}^{s*} = \text{argmin}_{\mathbf{x} \in \chi_s} c_s(\mathbf{x})$
- Application to CO problems
  - For the TSP,  $\mathbf{x} \in \{0,1\}^N$  is the indicator vector for selecting a subset form  $N$  edges
  - The cost of this subset is calculated as:  $\text{cost}_{\text{TSP}}(\mathbf{x}, s) = \sum_i x_i \cdot d_i^{(s)}$ , where  $d_i^{(s)}$  denotes the weight of  $i$ -th edge in problem instance  $s$ , and the  $\text{valid}(\cdot)$  part of formula ensures that  $\mathbf{x}$  is a tour that visits each node exactly once and returns to the starting node at the end.

## ● Combinatorial optimization as a probabilistic inference

- The authors assumed the optimal solution  $\mathbf{x}_s^*$  is available for each training instance  $s$ , and they optimized the model through supervised learning
  - Let  $\mathcal{S} = \{s_i\}_1^N$  be iid training samples for a type of CO problem, they aim to maximize the likelihood of optimal solutions, where the loss function  $L$  is defined as:
    - $L(\theta) = \mathbb{E}_{s \in \mathcal{S}} [-\log p_\theta(\mathbf{x}^{s*} | s)] = \mathbb{E}_{s \in \mathcal{S}} [-\log p_\theta(\mathbf{x}_0)]$  for brevity

## ● Diffusion Models in DIFUSCO

- The likelihood of diffusion model is formed as  $p_\theta(\mathbf{x}_0)$ 
  - $p_\theta(\mathbf{x}_0) := \int p_\theta(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T}$ , where  $\mathbf{x}_1, \dots, \mathbf{x}_T$  are latents of the same dimensionality as the data  $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
    - The joint distribution  $p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$  is learned reverse process
  - The forward process  $q(\mathbf{x}_{1:T} | \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1})$  gradually corrupts the data into noised latent variables
- Training is performed by optimizing the usual variational bound on negative log-likelihood:
  - $\mathbb{E}[-\log p_\theta(\mathbf{x}_0)] \leq \mathbb{E}_q \left[ -\log \frac{p_\theta(\mathbf{x}_{0:T})}{q_\theta(\mathbf{x}_{1:T} | \mathbf{x}_0)} \right] = \mathbb{E}_q [\sum_{t>1} D_{KL}[q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) || p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)] - \log p_\theta(\mathbf{x}_0 | \mathbf{x}_1)] + C$

## ● Discrete diffusion models with multinomial noises

- The forward process is defined as:  $q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \text{Cat}(\mathbf{x}_t; \mathbf{p} = \tilde{\mathbf{x}}_{t-1}\mathbf{Q}_t)$ , where  $\mathbf{Q}_t = \begin{bmatrix} 1 - \beta_t & \beta_t \\ \beta_t & 1 - \beta_t \end{bmatrix}$  is the transition probability matrix
  - $\tilde{\mathbf{x}} \in \{0,1\}^{N \times 2}$  is converted from the original vector  $\mathbf{x} \in \{0,1\}^N$  with one-hot vector per row; and  $\tilde{\mathbf{x}}\mathbf{Q}$  computes a row-wise vector-matrix product
- The denoising neural network is trained to predict the clean data  $p_\theta(\tilde{\mathbf{x}}_0|\mathbf{x}_t)$  and the reverse process is obtained by substituting the predicted  $\tilde{\mathbf{x}}_0$  as  $\mathbf{x}_0$ 
  - $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \sum_{\tilde{\mathbf{x}}} q(\mathbf{x}_{t-1}|\mathbf{x}_t, \tilde{\mathbf{x}}_0) p_\theta(\tilde{\mathbf{x}}_0|\mathbf{x}_t)$

## ● Continuous diffusion for discrete data

- They proposed to rescale the  $\{0,1\}$ -valued variables  $\mathbf{x}_0$  to the  $\{-1, 1\}$  domain as  $\hat{\mathbf{x}}_0$ , and then treat them as real values.
  - The forward process in cont. diffusion is defined as:  $q(\hat{\mathbf{x}}_t|\hat{\mathbf{x}}_{t-1}) := \mathcal{N}(\hat{\mathbf{x}}_t; \sqrt{1 - \beta_t}\hat{\mathbf{x}}_{t-1}, \beta_t\mathbf{I})$
- The denoising neural network is trained to predict the unscaled Gaussian noise  $\tilde{\epsilon}_t$ 
  - $\tilde{\epsilon}_t = \frac{(\hat{\mathbf{x}}_t - \sqrt{\alpha_t}\hat{\mathbf{x}}_0)}{\sqrt{1 - \alpha_t}} = f_\theta(\hat{\mathbf{x}}_t, t)$
- The reverse process can use a point estimation of  $\hat{\mathbf{x}}_0$  in the posterior:  $p_\theta(\hat{\mathbf{x}}_{t-1}|\hat{\mathbf{x}}_t) = q\left(\hat{\mathbf{x}}_{t-1}|\hat{\mathbf{x}}_t, \frac{\hat{\mathbf{x}}_t - \sqrt{1 - \alpha_t}f_\theta(\hat{\mathbf{x}}_t, t)}{\sqrt{\alpha_t}}\right)$
- For generating discrete data, after the continuous data  $\hat{\mathbf{x}}_t$  is generated, a thresholding/quantization operation is applied to convert them back to  $\{0,1\}$ -valued variables  $\mathbf{x}_0$  as the model's prediction

## ● Anisotropic graph neural network(AGNN)

- They adopt an anisotropic graph neural network which could produce the embeddings for both nodes and edges as the backbone network for diffusion models
  - The variables in the network output could be the states of either nodes, as in the case of MIS problems, or edges, as in the case of TSP
- Notation
  - $\mathbf{h}_i^\ell$ : node features at layer  $\ell$  associated with node  $i$
  - $\mathbf{e}_{ij}^\ell$ : edge features at layer  $\ell$  associated with edge  $ij$
  - $\mathbf{t}$ : the sinusoidal features of denoising timestep  $t$
  - $\mathbf{U}^\ell, \mathbf{V}^\ell, \mathbf{P}^\ell, \mathbf{Q}^\ell, \mathbf{R}^\ell \in \mathbb{R}^{d \times d}$ : learnable parameters of layer  $\ell$
  - $\mathcal{A}$ : aggregation function SUM pooling,  $\mathcal{N}_i$ : the neighborhoods of node  $i$
- The features at the next layer is propagated with an anisotropic message passing scheme:
  - $\hat{\mathbf{e}}_{ij}^{\ell+1} = \mathbf{P}^\ell \mathbf{e}_{ij}^\ell + \mathbf{Q}^\ell \mathbf{h}_i^\ell + \mathbf{R}^\ell \mathbf{h}_j^\ell$ ,  $\mathbf{e}_{ij}^{\ell+1} = \mathbf{e}_{ij}^\ell + \text{MLP}_e \left( \text{BN}(\hat{\mathbf{e}}_{ij}^{\ell+1}) \right) + \text{MLP}_t(\mathbf{t})$ ,
  - $\mathbf{h}_i^{\ell+1} = \mathbf{h}_i^\ell + \alpha \left( \text{BN} \left( \mathbf{U}^\ell \mathbf{h}_i^\ell + \mathcal{A}_{j \in \mathcal{N}_i} (\sigma(\hat{\mathbf{e}}_{ij}^{\ell+1}) \odot \mathbf{V}^\ell \mathbf{h}_j^\ell) \right) \right)$

## ● Decoding strategies for diffusion-based solvers

- After the training of the parameterized denoising network, the solutions are sampled for the diffusion models  $p_{\theta}(\mathbf{x}_0|s)$  for final evaluation
  - To sample the feasible solution that is suitable to the CO problems using the probabilistic generative model, they proposed specialized strategies
- Heatmap generation
  - They adapt the diffusion models to generate heatmaps by making the following appropriate modification
    - 1) For discrete diffusion, they removed the final score of  $p_{\theta}(\mathbf{x}_0 = 1|s)$  is preserved as the heatmap scores
    - 2) For continuous diffusion, they removed the final quantization and used  $0.5(\hat{\mathbf{x}}_0 + 1)$  as the heatmap scores
  - They noted that unlike the previous heatmap approaches that produce a single conditionally independent distribution for all variables, DIFUSCO could produce diverse multimodal output distributions by using different random seeds
- Solution sampling
  - They sample multiple heatmaps from  $p_{\theta}(\mathbf{x}_0|s)$  with different random seeds and then applying the greedy decoding algorithm to each heatmap

# Experimental Results

# Experimental settings

## ● Datasets

- They generated and labeled the training instances using exact solver(CONCORDE) for TSP-50/100 and heuristic solver(LKH-3) for TSP 500/1000/10000
  - They used sparse graphs by limiting each node to have only  $k$  edges to its nearest neighbors based on the Euclidian distances: they set  $k$  to 50 for TSP-500 and 100 for TSP-1000/10000.

## ● Model settings

- Diffusion related setting
  - They used  $T=1000$  denoising steps for the training model on all datasets.
  - Following previous studies, they use a simple linear noise schedule for  $\{\beta_t\}_{t=1}^T$ , where  $\beta_1 = 10^{-4}$  and  $\beta_T = 0.02$ .
- Decoding strategy
  - They utilized greedy decoding and 2-opt scheme as the default decoding scheme for experiments

## ● Evaluation metrics

- They provided three metrics: average tour length, average relative performance gap, and total run time

# Experiment results (I)

## ● Design analysis with TSP-50 and TSP-100

### ■ Discrete v.s. Continuous diffusion

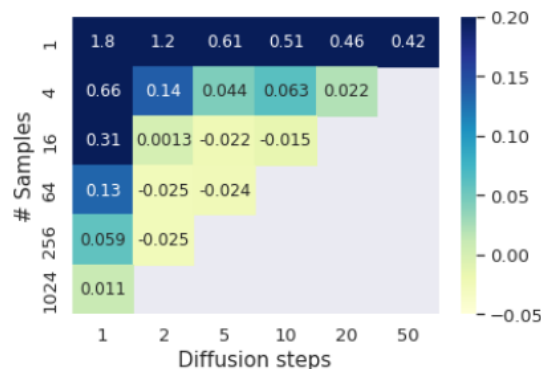
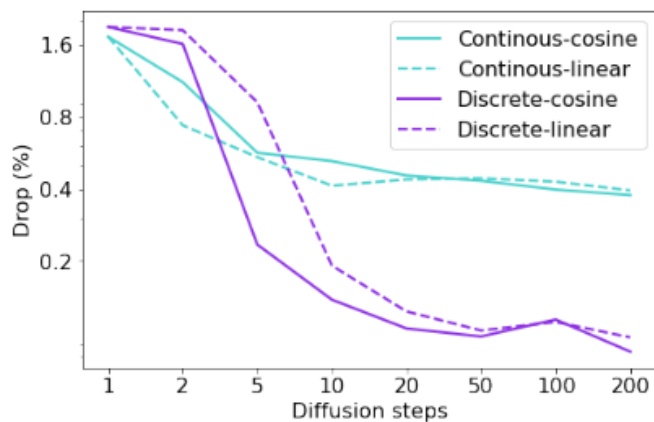
- They investigated the suitability of two diffusion approaches for CO and explored the effectiveness of different denoising schedules, such as linear and cosine schedule
- They demonstrated that discrete diffusion consistently outperforms the continuous diffusion by a large margin
  - Besides, the cosine schedule is superior to linear

### ■ More diffusion iterations v.s. More sampling

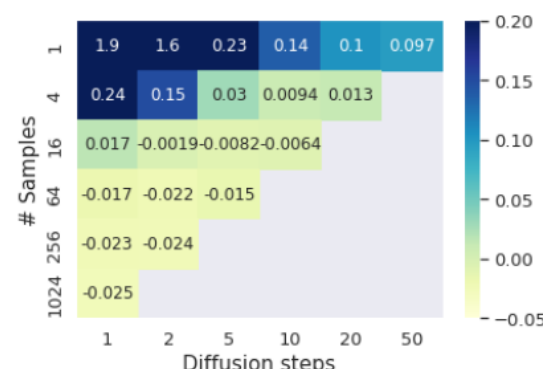
- They investigated the trade-off between the # of diffusion iterations and the # of samples for diffusion-based solvers
- They found that 50 (diffusion steps)  $\times$  1 (samples) policy and 10 (diffusion steps)  $\times$  16 (samples) policy make a good balance

| ALGORITHM       | TYPE                      | TSP-50      |              | TSP-100     |              |
|-----------------|---------------------------|-------------|--------------|-------------|--------------|
|                 |                           | LENGTH↓     | GAP(%)↓      | LENGTH↓     | GAP(%)↓      |
| CONCORDE*       | EXACT                     | 5.69        | 0.00         | 7.76        | 0.00         |
| 2-OPT           | HEURISTICS                | 5.86        | 2.95         | 8.03        | 3.54         |
| AM              | GREEDY                    | 5.80        | 1.76         | 8.12        | 4.53         |
| GCN             | GREEDY                    | 5.87        | 3.10         | 8.41        | 8.38         |
| TRANSFORMER     | GREEDY                    | 5.71        | 0.31         | 7.88        | 1.42         |
| POMO            | GREEDY                    | 5.73        | 0.64         | 7.84        | 1.07         |
| SYM-NCO         | GREEDY                    | -           | -            | 7.84        | 0.94         |
| DPDP            | 1k-IMPROVEMENTS           | 5.70        | 0.14         | 7.89        | 1.62         |
| IMAGE DIFFUSION | GREEDY <sup>†</sup>       | 5.76        | 1.23         | 7.92        | 2.11         |
| <b>Ours</b>     | <b>GREEDY<sup>†</sup></b> | <b>5.70</b> | <b>0.10</b>  | <b>7.78</b> | <b>0.24</b>  |
| AM              | 1k×SAMPLING               | 5.73        | 0.52         | 7.94        | 2.26         |
| GCN             | 2k×SAMPLING               | 5.70        | 0.01         | 7.87        | 1.39         |
| TRANSFORMER     | 2k×SAMPLING               | 5.69        | 0.00         | 7.76        | 0.39         |
| POMO            | 8×AUGMENT                 | 5.69        | 0.03         | 7.77        | 0.14         |
| SYM-NCO         | 100×SAMPLING              | -           | -            | 7.79        | 0.39         |
| MDAM            | 50×SAMPLING               | 5.70        | 0.03         | 7.79        | 0.38         |
| DPDP            | 100k-IMPROVEMENTS         | 5.70        | 0.00         | 7.77        | 0.00         |
| <b>Ours</b>     | <b>16×SAMPLING</b>        | <b>5.69</b> | <b>-0.01</b> | <b>7.76</b> | <b>-0.01</b> |

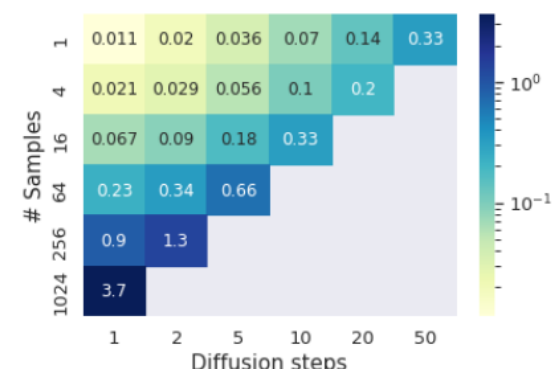
**Results on TSP-50 and TSP-100**



(a) Continuous diffusion



(b) Discrete diffusion



(c) Runtime

**Comparison of diffusion models**

**The performance Gap (%) for continuous diffusion (a) and discrete diffusion (b) and per-instance run time (s)**



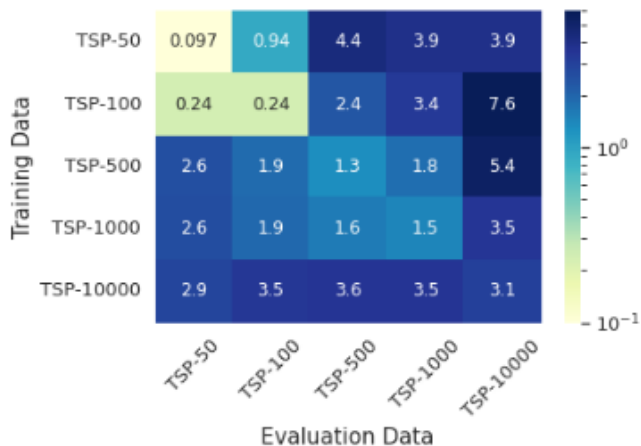
# Experiment results (II) – Quantitative results

## ● Main results on large-scale TSP

- They compared discrete DIFUSCO to other SOTA neural solvers on TSP problem across various scales.
  - The table on the right showed that most previous probabilistic solvers (except DIMES) becomes untrainable on TSP problems of these scales, so the results of these models are reported with TSP-100 trained models
- From the table, they provided that DIFUSCO strongly outperforms the previous neural solves on all settings

## ● Generalization Tests

- They also studied the generalization ability of DIFUSCO trained on a set of TSP
  - They showed that unlike the RL-trained or SL-trained methods, DIFUSCO has a strong generalization ability



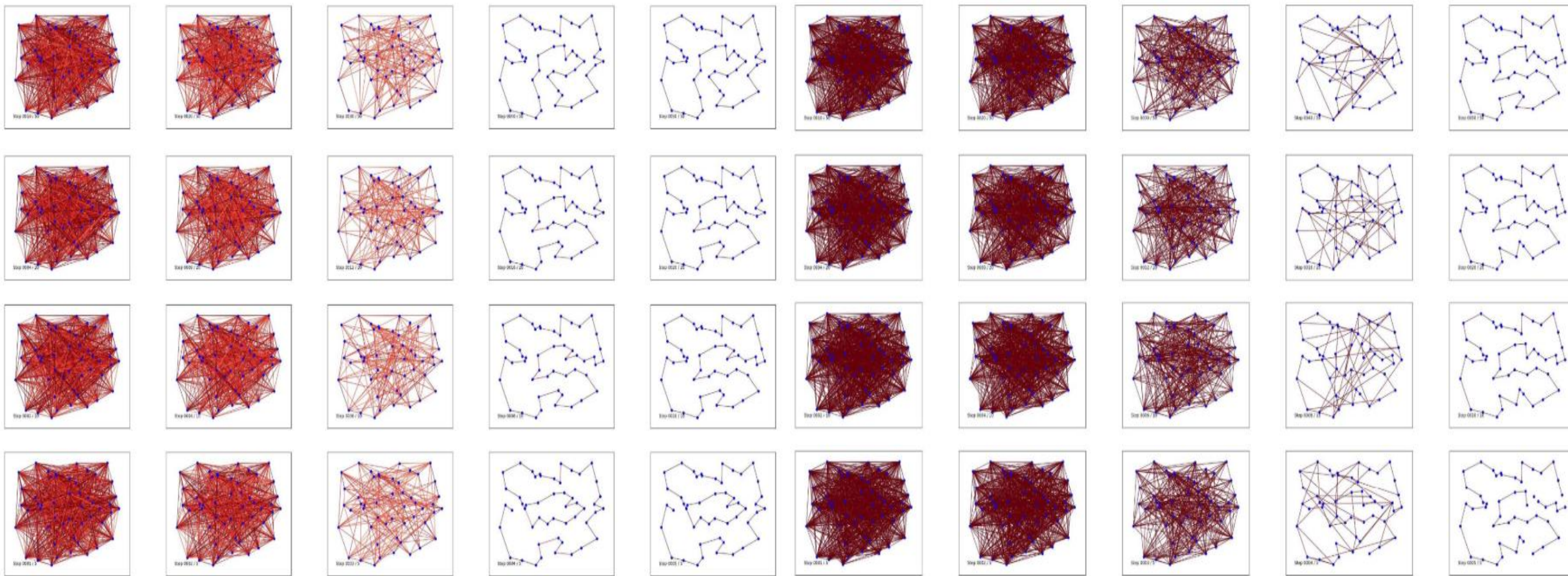
Generalization tests of DIFUSCO

| ALGORITHM           | TYPE        | TSP-500      |              |               | TSP-1000     |              |               | TSP-10000    |              |               |
|---------------------|-------------|--------------|--------------|---------------|--------------|--------------|---------------|--------------|--------------|---------------|
|                     |             | LENGTH ↓     | GAP ↓        | TIME ↓        | LENGTH ↓     | GAP ↓        | TIME ↓        | LENGTH ↓     | GAP ↓        | TIME ↓        |
| CONCORDE            | EXACT       | 16.55*       | —            | 37.66m        | 23.12*       | —            | 6.65h         | N/A          | N/A          | N/A           |
| GUROBI              | EXACT       | 16.55        | 0.00%        | 45.63h        | N/A          | N/A          | N/A           | N/A          | N/A          | N/A           |
| LKH-3 (DEFAULT)     | HEURISTICS  | 16.55        | 0.00%        | 46.28m        | 23.12        | 0.00%        | 2.57h         | 71.77*       | —            | 8.8h          |
| LKH-3 (LESS TRAILS) | HEURISTICS  | 16.55        | 0.00%        | 3.03m         | 23.12        | 0.00%        | 7.73m         | 71.79        | —            | 51.27m        |
| FARTHEST INSERTION  | HEURISTICS  | 18.30        | 10.57%       | 0s            | 25.72        | 11.25%       | 0s            | 80.59        | 12.29%       | 6s            |
| AM                  | RL+G        | 20.02        | 20.99%       | 1.51m         | 31.15        | 34.75%       | 3.18m         | 141.68       | 97.39%       | 5.99m         |
| GCN                 | SL+G        | 29.72        | 79.61%       | 6.67m         | 48.62        | 110.29%      | 28.52m        | N/A          | N/A          | N/A           |
| POMO+EAS-EMB        | RL+AS+G     | 19.24        | 16.25%       | 12.80h        | N/A          | N/A          | N/A           | N/A          | N/A          | N/A           |
| POMO+EAS-TAB        | RL+AS+G     | 24.54        | 48.22%       | 11.61h        | 49.56        | 114.36%      | 63.45h        | N/A          | N/A          | N/A           |
| DIMES               | RL+G        | 18.93        | 14.38%       | 0.97m         | 26.58        | 14.97%       | 2.08m         | 86.44        | 20.44%       | 4.65m         |
| DIMES               | RL+AS+G     | 17.81        | 7.61%        | 2.10h         | 24.91        | 7.74%        | 4.49h         | 80.45        | 12.09%       | 3.07h         |
| OURS (DIFUSCO)      | SL+G†       | 18.35        | 10.85%       | 3.61m         | 26.14        | 13.06%       | 11.86m        | 98.15        | 36.75%       | 28.51m        |
| OURS (DIFUSCO)      | SL+G†+2-OPT | <b>16.80</b> | <b>1.49%</b> | <b>3.65m</b>  | <b>23.56</b> | <b>1.90%</b> | <b>12.06m</b> | <b>73.99</b> | <b>3.10%</b> | <b>35.38m</b> |
| EAN                 | RL+S+2-OPT  | 23.75        | 43.57%       | 57.76m        | 47.73        | 106.46%      | 5.39h         | N/A          | N/A          | N/A           |
| AM                  | RL+BS       | 19.53        | 18.03%       | 21.99m        | 29.90        | 29.23%       | 1.64h         | 129.40       | 80.28%       | 1.81h         |
| GCN                 | SL+BS       | 30.37        | 83.55%       | 38.02m        | 51.26        | 121.73%      | 51.67m        | N/A          | N/A          | N/A           |
| DIMES               | RL+S        | 18.84        | 13.84%       | 1.06m         | 26.36        | 14.01%       | 2.38m         | 85.75        | 19.48%       | 4.80m         |
| DIMES               | RL+AS+S     | 17.80        | 7.55%        | 2.11h         | 24.89        | 7.70%        | 4.53h         | 80.42        | 12.05%       | 3.12h         |
| OURS (DIFUSCO)      | SL+S        | 17.23        | 4.08%        | 11.02m        | 25.19        | 8.95%        | 46.08m        | 95.52        | 33.09%       | 6.59h         |
| OURS (DIFUSCO)      | SL+S+2-OPT  | <b>16.65</b> | <b>0.57%</b> | <b>11.46m</b> | <b>23.45</b> | <b>1.43%</b> | <b>48.09m</b> | <b>73.89</b> | <b>2.95%</b> | <b>6.72h</b>  |
| ATT-GCN             | SL+MCTS     | 16.97        | 2.54%        | 2.20m         | 23.86        | 3.22%        | 4.10m         | 74.93        | 4.39%        | 21.49m        |
| DIMES               | RL+MCTS     | 16.87        | 1.93%        | 2.92m         | 23.73        | 2.64%        | 6.87m         | 74.63        | 3.98%        | 29.83m        |
| DIMES               | RL+AS+MCTS  | 16.84        | 1.76%        | 2.15h         | 23.69        | 2.46%        | 4.62h         | 74.06        | 3.19%        | 3.57h         |
| OURS (DIFUSCO)      | SL+MCTS     | <b>16.63</b> | <b>0.46%</b> | <b>10.13m</b> | <b>23.39</b> | <b>1.17%</b> | <b>24.47m</b> | <b>73.62</b> | <b>2.58%</b> | <b>47.36m</b> |

Results on large-scale TSP problems

# Experiment results (III) – Qualitative results

- Qualitative illustration of how diffusion steps affect the generation quality of continuous/discrete diffusion models



*Continuous DIFUSCO with 50, 20, 10, 5 diffusion steps in cosine schedule*

*Discrete DIFUSCO with 50, 20, 10, 5 diffusion steps in cosine schedule*

**Thank you!**

# Q&A